

PROJECT REPORT ON SMART ACCESS GATE SYSTEM USING ARDUINO UNO

SUBMITTED BY

Leitanthem Alex (233013)

Maisnam Pravin Singh (233014)

UNDER THE GUIDANCE OF

Mrs. Sharmila Meinam (Assistant Professor)

Department of Electronics and Communication Engineering

Manipur Institute of Technology, MANIPUR UNIVERSITY



MANIPUR INSTITUTE OF TECHNOLOGY

(A Constituent College of Manipur University)

ABSTRACT

The **Smart Access Gate System** is an automated, sensor-based gate control system designed to enable secure and efficient vehicle access in controlled areas. Utilizing an ultrasonic sensor to detect approaching vehicles, the system triggers a servo motor to automatically open the gate when a vehicle is within a set distance threshold. LED indicators and a buzzer provide visual and auditory alerts, while an LCD display communicates the gate's operational status to users. The gate closes automatically after a set delay if no additional vehicles are detected. Additionally, a manual override feature allows for controlled gate operation in case of sensor bypass or maintenance. This system minimizes the need for manual intervention, enhancing security and convenience in controlled-access environments.

CONTENTS

1. Introduction.....	4
2. System Design and Components.....	5-8
4. Circuit Diagram System Workflow.....	9-11
5. Flowchart (System Workflow)	12-14
6. Programming Logic.....	15
7. Code Explanation.....	16-19
8. Testing Procedure.....	20-22
9. Applications and Future Enhancements.....	23-24
10. Conclusion.....	25
11. References.....	26
12. Appendix.....	27-33

INTRODUCTION

Background

In modern infrastructure, automated access control systems play a crucial role in enhancing the management of entry and exit points across various settings, including parking lots, residential complexes, and industrial sites. The increasing need for secure, convenient, and efficient access solutions has led to a growing reliance on automation technologies. Automated gate control systems streamline vehicle access, minimizing the need for human intervention while ensuring a consistent level of security. By automating gate operations, these systems reduce operational costs, prevent unauthorized entry, and improve traffic flow within restricted zones, which is particularly valuable in high-traffic areas.

Objective

This project aims to design and implement a Smart Access Gate System that uses basic sensors and actuators to provide a cost-effective, automated solution for managing entry and exit points. By detecting the presence of vehicles and responding accordingly, the system offers a practical means of controlling access to restricted zones. The goal is to create an intuitive, automated gate that is easily adaptable to environments requiring efficient access management.

Scope

The scope of this project is a small-scale prototype designed primarily for learning and demonstration purposes. While this model functions effectively as a proof of concept, it could be adapted for real-world applications with further refinements, such as incorporating advanced sensors, remote monitoring capabilities, and enhanced security features. As a functional educational tool, this project demonstrates the principles of automated access control systems and their potential applications in modern infrastructure.

SYSTEM DESIGN AND COMPONENTS

Hardware Components Used

1. Ultrasonic Sensor

An ultrasonic sensor is used to detect the presence of vehicles approaching the gate by measuring the distance between the sensor and nearby objects. When a vehicle comes within a defined range, the sensor triggers the gate to open, allowing access. This non-contact sensing capability makes it reliable for outdoor use.



2. Servo Motor

A servo motor controls the physical movement of the gate, rotating to open and close it as instructed by the system. The motor's precision and control make it suitable for applications where specific angles of movement are

required, ensuring smooth and accurate gate operation.



3. LED Indicators

LED indicators provide visual cues to inform users of the gate's current state. A green LED signals the gates' opening, while a red LED indicates the gate is closing. These indicators enhance the system's usability by providing clear, real-time feedback.



4. Buzzer

The buzzer offers an audible warning before the gate opens or closes, alerting nearby individuals to the gate's movement. This is particularly useful in ensuring safety, as the sound warns of impending motion, preventing accidental contact with the gate.



5. I2C LCD Display

An LCD display with I2C communication is used to display status messages, such as "Opening Gate!" and "Closing Gate!" These messages provide clear and direct information on the gate's status, enhancing the system's user-friendliness and ensuring that users are aware of its operation.



6. Manual Override Switch

The manual override switch allows users to control the gate manually, bypassing the automated system. This feature is essential for maintenance or in emergency situations, providing a fail-safe option for gate operation when needed.



CIRCUIT DIAGRAM

1. Ultrasonic Sensor (HC-SR04)

- **VCC:** Connect to Arduino **5V**.
- **GND:** Connect to Arduino **GND**.
- **Trig Pin:** Connect to **Digital Pin 9** on the Arduino.
- **Echo Pin:** Connect to **Digital Pin 10** on the Arduino.

2. Servo Motor

- **Signal:** Connect to **Digital Pin 6** on the Arduino.
- **Power:** Connect to **5V**
- **Ground:** Connect to **GND**.

3. LED Indicators

- **Green LED:**
 - **Anode (longer leg):** Connect to **Digital Pin 7** with a **220-ohm resistor** in series.
 - **Cathode:** Connect to **GND**.
- **Red LED:**
 - **Anode (longer leg):** Connect to **Digital Pin 8** with a **220-ohm resistor** in series.
 - **Cathode:** Connect to **GND**.

4. Buzzer

- **Positive Terminal:** Connect to **Digital Pin 13**.
- **Negative Terminal:** Connect to **GND**.

5. I2C LCD Display (16x2)

- **VCC:** Connect to **5V**.
- **GND:** Connect to **GND**.
- **SDA:** Connect to **A4** (on Arduino Uno).
- **SCL:** Connect to **A5** (on Arduino Uno).

6. Manual Override Switch

- **One terminal:** Connect to **Analog Pin A0**.
- **Another terminal:** Connect to **GND**.

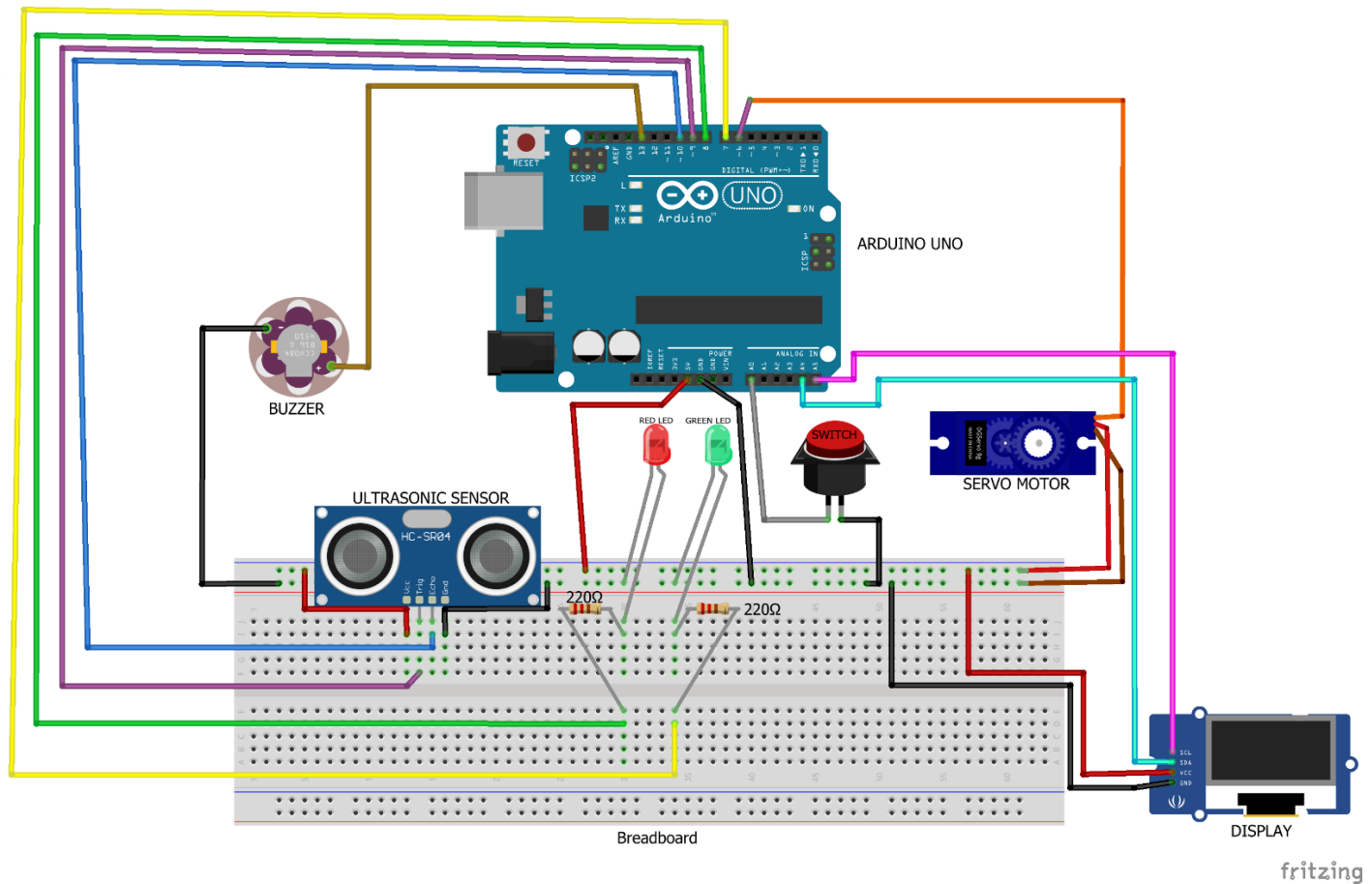


Figure 1. CIRCUIT DIAGRAM

Figure 1 shows the circuit diagram of the project, different color of wire for all the individual connections. The **red** wire indicates it is connected to the **5V supply** and the **black** wire indicates it is connected to the **ground**.

SYSTEM WORKFLOW

Vehicle Detection

- When a vehicle approaches within the set threshold distance (8 cm), the ultrasonic sensor measures the distance and detects the object.
- If the vehicle is detected, the sensor sends a signal to the Arduino microcontroller to initiate the gate-opening sequence.

Warning Alert

- The system sounds a brief warning on the buzzer to alert nearby individuals of the gate's impending movement.
- The green LED blinks to indicate that the gate is about to open, and the LCD displays the message "Opening Gate!" to provide visual feedback.

Gate Opening

- The servo motor receives a command from the microcontroller and rotates to the open position (90 degrees), allowing the vehicle to pass through.
- The green LED continues blinking while the gate is open.

Vehicle Exit Detection and Timer

- Once the vehicle has passed, the ultrasonic sensor detects the absence of any nearby objects.
- A timer begins counting down (set to 6 seconds in the code) during which the system waits for any additional vehicles.

Gate Closing

- If no vehicle is detected within the delay period, the red LED begins to blink, and the LCD displays the message "Closing Gate!".
- The servo motor rotates back to its closed position (0 degrees), and the red LED continues blinking during the closing process.

Manual Override

The manual override switch enables direct control over the gate, bypassing the automated sensor-based system. This switch is particularly useful during maintenance or in emergency situations where manual intervention is needed.

- Manual Control Activation
 - When pressed, the switch toggles the gate between open and closed states.
 - If the gate is closed, pressing the switch opens it, and if the gate is open, pressing the switch closes it.
- This mode temporarily disables the automatic sensor-based control, allowing for safe manual operation.

Figure 2 next page shows the flowchart of how the Arduino perform all the actions.

SYSTEM WORKFLOW

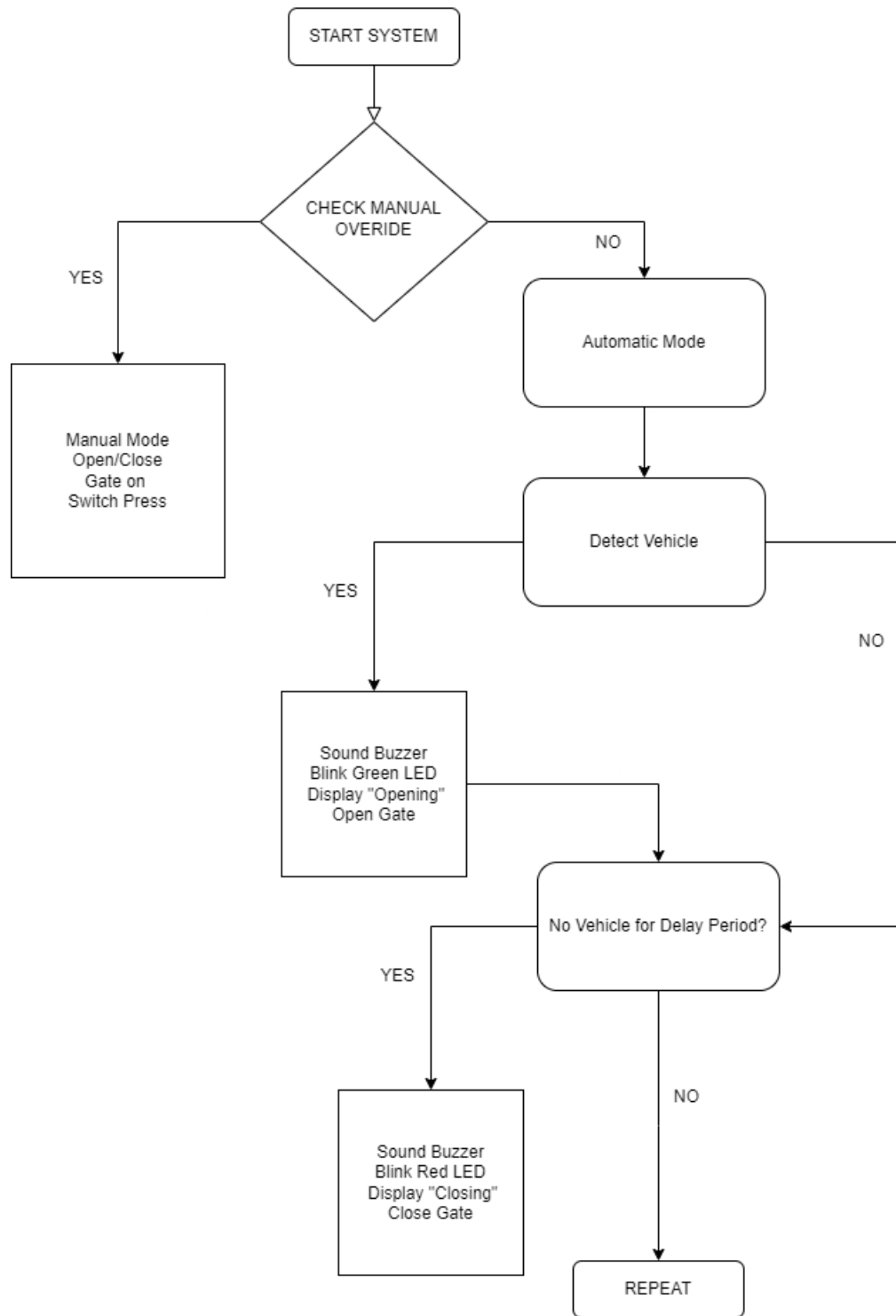


Figure 2. SYSTEM WORKFLOW (Flowchart)

PROGRAMMING LOGIC

1. Reading Distance from the Ultrasonic Sensor

- a. The ultrasonic sensor detects the presence of a vehicle by calculating the distance between the sensor and any object in front of it.
- b. The sensor sends a pulse, and the time taken for the pulse to reflect back is used to calculate the distance. If the distance is below the threshold (8 cm), the system considers a vehicle detected.

2. Controlling the Servo Motor for Gate Movement

- a. When the ultrasonic sensor detects a vehicle, the servo motor is activated to open the gate by rotating to 90 degrees.
- b. After a delay (if no vehicle is detected), the servo motor closes the gate by rotating back to 0 degrees.

3. Blinking LEDs and Displaying Messages on the LCD

- a. During the gate-opening sequence, the system blinks the green LED and displays “Opening Gate!” on the LCD.
- b. When closing the gate, the red LED blinks, and “Closing Gate!” is displayed on the LCD.
- c. A buzzer sounds briefly before each action to alert nearby individuals.

4. Handling Manual Override Mode

- a. A switch is used to override the automated system, allowing for manual gate control.
- b. When activated, the manual mode opens or closes the gate based on the current state, regardless of sensor readings.
- c. This mode is helpful for maintenance or emergency scenarios, and it disables automatic functionality while active.

CODE EXPLANATION

Reading Distance from the Ultrasonic Sensor: This section calculates the distance from an object based on the time it takes for an ultrasonic pulse to return. It then checks if this distance is within the threshold.

```
void automaticGateControl() {
    long duration, distance;
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    duration = pulseIn(echoPin, HIGH);
    distance = duration * 0.034 / 2; // Convert to cm

    if (distance <= distanceThreshold) {
        if (currentAngle == closeAngle) {
            digitalWrite(buzzerPin, HIGH);
            delay(warningTime);
            digitalWrite(buzzerPin, LOW);

            lcd.clear();
            lcd.print("Opening Gate!");
            blinkLED(greenLEDPin, 3000);
            openGate();
        }
        lastDetectionTime = millis();
    }
}
```

Explanation

1. The ultrasonic sensor triggers a pulse and measures the duration it takes to return.
2. The distance is calculated and checked against the `distanceThreshold` of 8 cm.
3. If within range, the system prepares to open the gate by sounding the buzzer, displaying a message, and calling `openGate()`.

Controlling the Servo Motor for Gate Movement: These functions handle smooth gate movement by incrementing the servo angle in steps.

```
void openGate() {  
    for (int angle = currentAngle; angle <= openAngle; angle += angleStep) {  
        gateServo.write(angle);  
        delay(500);  
    }  
    currentAngle = openAngle;  
}
```

```
void closeGate() {  
    for (int angle = currentAngle; angle >= closeAngle; angle -= angleStep) {  
        gateServo.write(angle);  
        delay(500);  
    }  
    currentAngle = closeAngle;  
}
```

Explanation:

- **openGate()** smoothly rotates the servo to open the gate by increasing the angle in steps (30 degrees).
- **closeGate()** similarly reduces the angle to smoothly close the gate.
- **currentAngle** stores the gate's position, so the system knows its state.

Blinking LEDs and Displaying Messages on the LCD: The following function blinks an LED and displays messages at different stages of gate operation.

```
void blinkLED(int ledPin, int duration) {
    unsigned long blinkStartTime = millis();
    while (millis() - blinkStartTime < duration) {
        digitalWrite(ledPin, HIGH);
        delay(250);
        digitalWrite(ledPin, LOW);
        delay(250);
    }
}
```

Explanation:

- **blinkLED()** alternates an LED between on and off states every 250 ms, making it blink.
- The function is called to blink the green LED when opening and the red LED when closing, providing a visual indicator of gate movement.

Handling Manual Override Mode: This function detects if the manual override switch is pressed, toggling manual gate control.

```
void manualGateControl() {
    if (!isManualOverrideActive) {
        isManualOverrideActive = true;
        openGate();
    } else {
        isManualOverrideActive = false;
        closeGate();
    }

    while (digitalRead(switchPin) == LOW) {
        delay(100);
    }
}
```

Explanation

- When the switch is pressed, the function toggles **isManualOverrideActive**, opening or closing the gate based on the current state.
- This disables automatic gate control, allowing for manual operation as long as the override switch is active. The system waits for the switch to be released before checking for further inputs.

TESTING PROCEDURE

1. Component Testing:

a. Ultrasonic Sensor:

- Tested by positioning objects at varying distances to confirm accurate readings below and above the 8 cm threshold.
- Observed changes in readings on the Serial Monitor to verify correct detection distances.

b. Servo Motor:

- Tested independently by sending angle commands through the Arduino code to verify smooth and accurate rotation.
- Checked both 0 degrees (closed position) and 90 degrees (open position) to ensure the gate moved correctly.

c. LED Indicators:

- Each LED (green and red) was tested by toggling it on and off in the code.
- Verified that LEDs blinked as intended during the gate opening (green) and closing (red) processes.

d. Buzzer:

- The buzzer was tested by setting it to emit a brief sound in response to vehicle detection and gate movement.
- Confirmed the buzzer provided an audible alert before both opening and closing events.

e. LCD Display:

- Display messages were tested by printing sample text to ensure clarity and functionality.

- Verified that “Opening Gate!” and “Closing Gate!” messages displayed correctly and in sync with gate movement.

2. Assembled System Testing:

- After each component was tested, the entire system was assembled to observe interactions.
- Vehicles or objects were positioned at a close range to trigger the sensor, verifying the complete cycle of detecting, alerting, opening, waiting, and closing.
- The system was tested in both automatic and manual override modes to ensure it could seamlessly switch between them.

PERFORMANCE

- **Vehicle Detection:**
 - The ultrasonic sensor consistently detected vehicles within the threshold, activating the gate-opening sequence as expected.
 - The system responded to approaching objects with minimal delay, accurately distinguishing between vehicle presence and absence.
- **Gate Opening and Closing:**
 - The servo motor reliably executed opening and closing at the correct times, following both sensor triggers and manual overrides.
 - Gate motion was smooth, and step increments of 30 degrees provided controlled transitions that aligned well with LED indicators.
- **LCD Display and Alerts:**
 - The LCD displayed messages in real time, accurately indicating "Opening Gate!" and "Closing Gate!" based on the gate state.

- The buzzer and LED indicators gave clear audio-visual alerts to signal nearby individuals, enhancing operational feedback.

CHALLENGES

1. Timing Adjustments:

- a. Challenge:** Finding an appropriate delay between object detection and gate movement to avoid premature or delayed actions.
- b. Solution:** Fine-tuned the delay for the ultrasonic sensor and adjusted the LED blink durations to match the gates' opening and closing speed.

2. Component Compatibility:

- a. Challenge:** Ensuring all components (e.g., servo motor and LEDs) operated within power limits to avoid interference.
- b. Solution:** Added a delay in servo operation to smoothen transitions and minimize power fluctuations, which improved reliability.

3. Manual Override Functionality:

- a. Challenge:** Avoiding interference between automatic and manual modes when switching control methods.
- b. Solution:** Added a check for the manual override flag in the code, ensuring the system only responded to one mode at a time, preventing conflicts.

APPLICATION AND FUTURE ENHANCEMENTS

1. Implementation of RFID Technology:

- Future versions of the Smart Access Gate System will integrate RFID technology to facilitate efficient and secure access control. Users can be provided with RFID tags or key fobs, allowing for seamless entry and exit without the need to manually operate the gate. This implementation can significantly reduce wait times and enhance user convenience. Additionally, RFID readers can log entry and exit data, enabling better tracking of authorized access while preventing unauthorized entry.

2.Integration with Keypad Access Control:

- Alongside RFID, a keypad entry system can be included as a backup access method. This dual approach ensures that authorized users have multiple ways to access the system, improving reliability and convenience.

3. Security Camera Integration:

- Adding cameras for license plate recognition will enhance security by automatically logging vehicle entries associated with RFID tags. This system can identify unauthorized vehicles and monitor access in real time.

4. Remote Monitoring:

- The implementation of Wi-Fi or Bluetooth modules will allow users to control and monitor the gate remotely via a smartphone app or web interface. Users will receive notifications about gate status and access logs, providing them with real-time information.

5. Enhanced Security Features:

- Additional sensors will be integrated to detect unauthorized entry attempts. For instance, motion sensors can trigger alarms if someone tries to tamper with the gate or gain access without permission.

6. Integration with Smart Home Systems:

- Future iterations will also include compatibility with smart home platforms, enabling voice control and integration into a centralized smart home management system.

7. Data Analytics:

- Incorporating data analytics will allow users to analyze traffic patterns, peak entry times, and gate usage. This data can aid in optimizing the access control system and improving overall management.

CONCLUSION

In summary, the objective of our Smart Access Gate System project was to develop a small-scale prototype that demonstrates the principles of automated access control. This goal was achieved by integrating various components, including ultrasonic sensors for distance measurement, a servo motor for gate operation, and a **16x2 I2C LCD** display for user feedback. The system effectively illustrates how sensors and actuators work together to create an automated solution for managing access, showcasing the fundamentals of access control technology.

Throughout this project, we both developed valuable skills and gained knowledge in several key areas:

1. **Sensor Integration:**

- a. We learned how to integrate ultrasonic sensors into the Arduino platform, enabling accurate distance measurement to control the gate's operation. This experience deepened our understanding of how sensors interact with microcontrollers to gather real-time data.

2. **Coding for Hardware Control:**

- a. Writing code for the Arduino involved programming the logic for sensor readings and motor control. We gained proficiency in Arduino programming, including effectively using libraries and functions to manage hardware components and respond to sensor input.

3. **System Troubleshooting:**

- a. Throughout the project, we encountered various challenges, such as debugging hardware connections and optimizing code performance. This hands-on experience improved our problem-solving skills and taught us how to systematically identify and resolve issues in embedded systems.

Overall, this project has not only served as a proof of concept for automated access control systems but also provided a comprehensive learning experience for both of us. It has equipped us with practical skills that are essential for future endeavors in electronics and embedded systems, laying a solid foundation for further exploration in the field of automation and smart technologies.

REFERENCES

1. Margolis, M., & Smith, D. (2011). *Arduino Cookbook*. O'Reilly Media.
2. McRoberts, M. (2012). *Learning Arduino*. McGraw-Hill Education.
3. Choudhary, A., & Kumar, R. (2021). A Review on RFID Technology and its Applications. *International Journal of Computer Applications*, 174(1), 1-6. <https://doi.org/10.5120/ijca2021921334>
4. Samaniego, M., & Rojas, M. (2016). Smart Gate Control System Based on RFID and Arduino. *IEEE Latin America Transactions*, 14(5), 2261-2266. <https://doi.org/10.1109/TLA.2016.7493358>
5. Arduino Official Website. (n.d.). Arduino Documentation. Retrieved from <https://www.arduino.cc/en/Reference/HomePage>
6. Adafruit Learning System. (n.d.). Adafruit Tutorials. Retrieved from <https://learn.adafruit.com/>
7. Understanding Access Control Systems. (n.d.). *Security Info Watch*. Retrieved from <https://www.securityinfowatch.com>
8. The Role of Sensors in Smart Systems. (n.d.). *Electronics Tutorials*. Retrieved from <https://www.electronicwings.com>
9. ISO/IEC 14443. (2008). International Organization for Standardization.

APPENDIX

Source Code

The complete source code for the Smart Access Gate System project is provided below. This code is written for the Arduino platform and includes the necessary libraries and functions for integrating the ultrasonic sensor, servo motor, and I2C LCD display.

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <Servo.h>


// Define the pins for the ultrasonic sensor
const int trigPin = 9;
const int echoPin = 10;


// Define the pin for the servo motor
Servo gateServo;


// Define the distance threshold (adjusted to 8 cm)
const int distanceThreshold = 8; // 8 cm


// Timing variables
unsigned long lastDetectionTime = 0;    // Store the last time an object was detected
const unsigned long noObjectDelay = 6000; // Wait 6 seconds without detection before closing
const unsigned long warningTime = 1000; // Buzzer warning time (1 second)


// Gate position control variables
int currentAngle = 0; // Current angle of the gate (starts closed)
int openAngle = 90;   // The angle for the gate to be fully open
```

```

int closeAngle = 0;    // The angle for the gate to be fully closed

int angleStep = 30;    // Step size for each cycle (30 degrees)


// LEDs

const int greenLEDPin = 7; // Pin for the green LED (Gate opening)
const int redLEDPin = 8;   // Pin for the red LED (Gate closing)


// Buzzer

const int buzzerPin = 13; // Pin for the buzzer


// Switch for manual override

const int switchPin = A0; // Pin for the manual switch


// Manual mode flag

bool isManualOverrideActive = false; // Tracks whether the manual override is active


// Initialize the I2C LCD display (replace 0x27 with the correct address if necessary)
LiquidCrystal_I2C lcd(0x27, 16, 2);


void setup() {
    // Initialize the servo and set it to the closed position
    gateServo.attach(6);      // Attach the servo to pin 6
    gateServo.write(currentAngle); // Start with gate closed (0 degrees)


    // Initialize the ultrasonic sensor pins
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);


    // Initialize the LEDs
    pinMode(greenLEDPin, OUTPUT);

```

```

pinMode(redLEDPin, OUTPUT);
digitalWrite(greenLEDPin, LOW); // Initially OFF
digitalWrite(redLEDPin, LOW); // Initially OFF

// Initialize the buzzer
pinMode(buzzerPin, OUTPUT);
digitalWrite(buzzerPin, LOW); // Buzzer OFF initially

// Initialize the switch
pinMode(switchPin, INPUT_PULLUP); // Use internal pull-up resistor

// Initialize the LCD
lcd.init(); // Initialize the LCD
lcd.backlight(); // Turn on the backlight
lcd.clear();
lcd.print("System Ready"); // Initial message
delay(2000);
lcd.clear();
}

void loop() {
  // Check if manual switch is pressed
  if (digitalRead(switchPin) == LOW) {
    manualGateControl(); // Handle the manual override mode
  } else {
    if (!isManualOverrideActive) {
      automaticGateControl(); // Continue in automatic mode if manual override is inactive
    }
  }
}

```

```

    delay(100); // Short delay before the next reading
}

// Function for manual gate opening and override
void manualGateControl() {
    // Toggle the manual override flag on button press
    if (!isManualOverrideActive) {
        isManualOverrideActive = true;
        openGate(); // Open the gate on the first button press
    } else {
        isManualOverrideActive = false;
        closeGate(); // Close the gate on the second button press
    }

    // Wait for the button to be released to avoid repeated triggering
    while (digitalRead(switchPin) == LOW) {
        delay(100); // Small delay
    }
}

// Function for automatic gate control
void automaticGateControl() {
    // Calculate distance using the ultrasonic sensor
    long duration, distance;
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    duration = pulseIn(echoPin, HIGH);

```

```

distance = duration * 0.034 / 2; // Convert to cm

// If the car is detected within the threshold (8 cm or less)
if (distance <= distanceThreshold) {
    // If the gate is currently closed, initiate the open process
    if (currentAngle == closeAngle) {
        // Sound the buzzer for 1 second before opening
        digitalWrite(buzzerPin, HIGH);
        delay(warningTime);
        digitalWrite(buzzerPin, LOW);

        // Display "Opening Gate!" while the green LED blinks
        lcd.clear();
        lcd.print("Opening Gate!");
        blinkLED(greenLEDPin, 3000); // Blink green LED for 3 seconds

        openGate(); // Open the gate immediately after LED blinking
    }

    // Reset the last detection time to the current time
    lastDetectionTime = millis();
}

// Check if no object has been detected for more than 6 seconds
if (millis() - lastDetectionTime > noObjectDelay && currentAngle == openAngle) {
    // Sound the buzzer for 3 seconds before closing
    digitalWrite(buzzerPin, HIGH);
    delay(3000);
    digitalWrite(buzzerPin, LOW);
}

```

```

// Display "Closing Gate!" while the red LED blinks
lcd.clear();
lcd.print("Closing Gate!");
blinkLED(redLEDPin, 3000); // Blink red LED for 3 seconds

closeGate(); // Close the gate after the LED blinks
}
}

// Function to blink an LED for a given duration (in milliseconds)
void blinkLED(int ledPin, int duration) {
    unsigned long blinkStartTime = millis();
    while (millis() - blinkStartTime < duration) {
        digitalWrite(ledPin, HIGH); // Turn LED ON
        delay(250);                 // Wait 250ms
        digitalWrite(ledPin, LOW);  // Turn LED OFF
        delay(250);                 // Wait 250ms
    }
}

// Function to open the gate smoothly (30-degree increments)
void openGate() {
    for (int angle = currentAngle; angle <= openAngle; angle += angleStep) {
        gateServo.write(angle);
        delay(500); // Small delay for smooth movement
    }
    currentAngle = openAngle;
}

// Function to close the gate smoothly (30-degree increments)

```

```
void closeGate() {  
  for (int angle = currentAngle; angle >= closeAngle; angle -= angleStep) {  
    gateServo.write(angle);  
    delay(500); // Small delay for smooth movement  
  }  
  currentAngle = closeAngle;  
}
```